240104004006
Zeynep Özge TOPTAŞ
LAB11 Report

I chose “conditional watch” command.
It must track pc, variables, halt flag, target index and the condition.
It must check conditions after each step. Because after each execution the variables’s value is changing, this means if we check before the step it would mean we’re checking the old value.
Execution continues if the value does not meet the condition and the program is not halted.
execution stops when value reaches the condition or when the debugger reaches the halt.

SET target_index to 0 (the index of the variable to watch)

SET condition_target to 10 (the value to compare against)

SET is_program_running to TRUE

WHILE is_program_running is TRUE:

1. READ current instruction using the Program Counter (pc)
2. EXECUTE the current instruction:
   - Update the variables (vars array)
   - Update the pc to the next instruction
   - Check if the instruction was HALT
3. OBSERVE the current value of vars[target_index]
4. CHECK CONDITION (After the step):

   IF the value of vars[target_index] is GREATER THAN condition_target:

      SET is_program_running to FALSE (stop execution)

   ELSE IF the program has reached a HALT instruction:

      SET is_program_running to FALSE (stop execution)

   ELSE:

      CONTINUE to the next iteration (execution continues)

Brief Explanation : My design extends the debugger we coded in labwork by adding a specific logic gate to the execution loop. In the lab, we stopped whenever a variable changed at all; in this design, the debugger only stops when the variable reaches a specific target value. This allows the user to run the program until a specific data is reached, making the debugging process more efficient.

The debugger command I designed works by observing the value of a specific variable after each

instruction and stopping execution when it reaches a target value the which was chosen by the user.